

40dB

Sistema de Monitoreo y Gestión de Contaminación Acústica en Tiempo Real.

Integrantes del grupo

Joaquín Meléndez
Ignacio Saavedra

Profesor a cargo:
Cristian Calderón

Fecha de entrega:
24-05-2026

Indice

Indice.....	2
Introducción	3
Contexto del proyecto.....	5
2.1 Antecedentes y motivación	5
2.2 Origen del problema	5
2.3 Marco normativo en Chile.....	5
2.4 Estado del arte	6
<i>Descripción del problema u oportunidad</i>	9
Problema central.....	9
Análisis de causas y efectos.....	9
<i>Descripción del mercado objetivo.....</i>	11
Objetivo general y objetivos específicos	13
Situación inicial del cliente (AS-IS)	14
Proceso actual de gestión del ruido en una municipalidad.....	14
Puntos de dolor.....	14
Planificación	15
Servicios Cloud a utilizar (IaaS / PaaS / SaaS)	17
Conceptualización (solución propuesta – versión preliminar).....	19
Alcance, supuestos y restricciones.....	21
Metodología de desarrollo y gestión del proyecto	23
Documentos y diagramas de diseño de la solución	26
Arquitectura de software y patrones de diseño	28
Tecnologías, lenguaje y dependencias (stack definitivo).....	30
Configuración de servidor de producción (paso a paso).....	32
Estado de avance del desarrollo (evidencia de producto funcional)	35
Integraciones necesarias.....	37
Bibliografía.....	40

Introducción

La contaminación acústica en zonas urbanas densamente pobladas se ha consolidado como un problema de salud pública con consecuencias documentadas sobre el sueño, la salud cardiovascular y el bienestar general de la población. Según datos del Ministerio del Medio Ambiente, más de 1,2 millones de habitantes del Gran Santiago están expuestos a niveles de ruido diurno por sobre las recomendaciones de la OCDE, cifra que asciende a 1,8 millones en horario nocturno. A pesar de la magnitud del problema, las herramientas con que cuentan las municipalidades chilenas para diagnosticarlo y actuar frente a él siguen siendo limitadas: mapas estáticos actualizados cada varios años, denuncias reactivas tramitadas caso a caso y mediciones puntuales en terreno que no permiten construir una visión continua del territorio.

El proyecto 40dB propone llenar ese vacío mediante una plataforma web de monitoreo colaborativo de ruido urbano que combina dos fuentes de datos complementarias: reportes ciudadanos enviados desde el navegador, con captura de decibeles desde el micrófono del dispositivo y geolocalización, y lecturas periódicas de sensores IoT basados en ESP32 instalados en puntos fijos de la comuna. Ambas fuentes alimentan un backend que cruza la información geográfica y temporal para asignar un nivel de confiabilidad a cada reporte, y un dashboard municipal que visualiza el resultado en un mapa de calor interactivo con filtros por zona, horario y tipo de fuente sonora. La solución está orientada a municipalidades como herramienta de gestión ambiental —en una primera etapa, a la Municipalidad de Maipú— permitiéndoles tomar decisiones basadas en datos empíricos continuos en lugar de depender exclusivamente de modelaciones teóricas o denuncias puntuales.

El presente informe documenta el diseño y el estado de avance del desarrollo del proyecto. Comienza por el contexto del problema, su marco normativo en Chile y el estado del arte de soluciones existentes a nivel nacional e internacional, para luego definir el problema central junto con su análisis de causas y efectos. Continúa con la descripción del mercado objetivo —municipalidades como cliente y vecinos como usuarios finales— y con el planteamiento de los objetivos generales y específicos que guían el desarrollo. A partir de allí se describe la situación inicial del cliente (AS-IS), la planificación del proyecto en seis sprints de dos semanas, los servicios cloud seleccionados (Render, Vercel, Supabase y HiveMQ Cloud) y la conceptualización preliminar de la solución (TO-BE), seguida del alcance, los supuestos y las restricciones que delimitan el trabajo. Las secciones posteriores profundizan en los aspectos de ingeniería: la metodología ágil adoptada por el equipo y la distribución de responsabilidades, los documentos y diagramas de diseño que estructuran la solución (casos de uso, flujos AS-IS y TO-BE, mockups, modelo de datos y arquitectura), la arquitectura de

software hexagonal lite junto con los patrones de diseño aplicados, el stack tecnológico definitivo y la configuración del servidor de producción paso a paso. Finalmente, se presenta el estado de avance del desarrollo con evidencia de producto funcional, las integraciones necesarias con servicios externos y una conclusión con los beneficios esperados y los próximos pasos del semestre.

Contexto del proyecto

2.1 Antecedentes y motivación

El ruido urbano es reconocido internacionalmente como un contaminante con impactos directos en la salud: trastornos del sueño, estrés, hipertensión y pérdida auditiva. La OMS estima la pérdida de al menos un millón de años de vida saludable anuales solo en Europa debido al ruido del tráfico. En Chile, la Superintendencia del Medio Ambiente (SMA) reporta que aproximadamente el 52% de las denuncias ambientales que recibe corresponden a contaminación acústica, y solo entre enero y marzo de 2025 ingresaron más de 1.100 denuncias por ruido, superando el promedio histórico del mismo período.

La motivación de este proyecto nace de la experiencia cotidiana de vivir en una ciudad ruidosa y de la constatación de que, pese a la existencia de normativa (DS 38/2011 del MMA, ordenanzas municipales), la fiscalización y el diagnóstico del problema carecen de herramientas tecnológicas accesibles y en tiempo real a nivel comunal.

2.2 Origen del problema

El problema se detectó por observación directa y experiencia personal: la percepción de niveles de ruido elevados en entornos urbanos cotidianos contrasta con la escasez de datos empíricos accesibles que permitan dimensionar y localizar el problema a escala de barrio o comuna. Actualmente, un vecino que sufre ruido molesto solo puede presentar una denuncia ante la SMA o la municipalidad, pero no tiene acceso a datos históricos ni en tiempo real de los niveles de ruido en su zona.

2.3 Marco normativo en Chile

La regulación del ruido en Chile se estructura en varios niveles. El **Decreto Supremo N°38/2011 del MMA** establece niveles máximos de emisión de ruido para fuentes fijas (industrias, comercio, construcción), con límites que varían según la zona territorial y el horario (diurno/nocturno). Por ejemplo, en zonas residenciales el límite nocturno es de 45 dB. Sin embargo, este decreto no regula fuentes móviles como el tránsito vehicular ni actividades propias de viviendas. Complementariamente, cada municipalidad puede dictar **ordenanzas locales** sobre ruidos molestos que cubren estos vacíos. A nivel nacional, se encuentra en

elaboración una **Norma Primaria de Calidad Ambiental para Ruido**, que apunta a establecer estándares de calidad del aire acústico (no solo de emisión).

En febrero de 2026 se publicó el **Decreto Supremo N°14/2024 del MMA**, que actualiza el DS 38/2011 con exigencias más estrictas.

2.4 Estado del arte

En Chile:

- **Mapa de Ruido del Gran Santiago (MMA/UACH):** Modelación por software del ruido vehicular en 35 comunas, actualizado en 2023. Es una herramienta de diagnóstico estática que se actualiza cada varios años y no captura datos en tiempo real ni fuentes de ruido incidentales (construcciones, eventos, etc.).
- **Mapa de Denuncias de Ruido (SMA, 2025):** Plataforma pública lanzada en la plataforma IDE de la SMA que visualiza denuncias históricas por comuna desde 2013 a 2024. Es reactivo (solo registra lo que se denuncia) y no mide niveles de ruido.
- **Red de monitoreo piloto del MMA:** Estaciones con sonómetros en puntos específicos de Santiago para medir la calidad del aire en tiempo real. Cobertura muy limitada y no accesible como plataforma pública interactiva. Tiene un símil con 40 dB en el sentido de utilizar sensores que reportan continuamente un estado del ambiente, en este caso es calidad del aire reportada a través de datos, nosotros queremos reportar cantidad de ruido a través de un heatmap.
- **SOSAFE:** Red social ciudadana chilena con 2 millones de usuarios aprox y convenios con más de 30 municipios, Carabineros y Bomberos. Permite reportar incidentes geolocalizados en categorías como robos, emergencias y ruidos molestos. Su foco es seguridad general, no acústica: los reportes de ruido son textuales y se derivan a seguridad municipal, sin medir decibeles ni generar mapas de ruido. Modelo freemium con funciones limitadas en versión gratuita.

A nivel internacional:

- **The Noise App (Reino Unido):** App de RHE Global especializada en denuncias de ruido vecinal, operativa en UK, Países Bajos, Irlanda y Australia. Modelo B2G: gratuita para ciudadanos si su municipio está suscrito. Permite grabar clips de audio/video de 30 segundos como evidencia, admisible en tribunales. Es una herramienta de gestión de denuncias, no de monitoreo: no mide dB continuamente ni genera mapas acústicos.

- **Alerta Surco (Perú):** App municipal del distrito de Santiago de Surco (Lima), lanzada en 2023, con más de 65 mil usuarios. Botón de pánico para solicitar serenazgo, bomberos o ambulancia con respuesta en 5 minutos, mostrando el recorrido de la unidad en mapa. Enfocada en seguridad y emergencias; anteriormente incluía categoría de ruidos molestos, pero fue eliminada. Alcance estrictamente distrital.
- **NoiseCapture (Francia):** Aplicación Android open-source para medición colaborativa de ruido ambiental. Genera mapas participativos con datos de smartphones. Solo Android, orientada a investigación, sin presencia activa en Latinoamérica.

Conclusión del análisis: No existe en Chile una plataforma que combine monitoreo ciudadano en tiempo real con sensores IoT fijos, orientada a municipalidades como herramienta de gestión. 40dB ocupa ese espacio.

Tabla de homologación:

Solución	Qué hace	Ventajas	Desventajas	Qué hará 40dB distinto
Mapa de Ruido MMA	Modela ruido vehicular por software en el Gran Santiago	Cobertura de 35 comunas, respaldo institucional	Estático, se actualiza cada año, no captura ruido incidental	Datos en tiempo real múltiples fuentes de ruido., granularidad comunal
Mapa Denuncias SMA	Visualiza denuncias históricas por comuna	Acceso público, cobertura nacional	Solo denuncias (reactivo), no mide dB, no es en tiempo real	Monitoreo proactivo con medición de dB + contexto denuncia ciudadana (proactivo + reactivo)
SOSAFE	Medio de	Acceso	Solo genera	Cruzar datos

	reportes en incidentes geolocalizados	público, reportes en tiempo real	reportes de usuarios, no contrasta con datos reales inmediatos.	reales con los reportes, será exclusivamente de ruido
The Noise App	App especializada en reportes de ruido	Permite grabar clips de audios para respaldar la denuncia.	No está disponible en Chile e incluye evidencia únicamente del denunciante.	Datos externos al usuario que reporta, dando mayor fiabilidad a la denuncia.

Descripción del problema u oportunidad

Problema central

Actualmente las municipalidades chilenas carecen de herramientas tecnológicas para monitorear el ruido urbano de forma continua y geolocalizada, lo que les impide diagnosticar zonas críticas con datos empíricos y tomar decisiones informadas de gestión ambiental.

Análisis de causas y efectos

El análisis se muestra presenta mediante este texto como también en un diagrama Ishikawa¹.

Causas principales:

1. **Tecnológicas:** No existen plataformas de monitoreo de ruido en tiempo real accesibles para municipios chilenos.
2. **Datos:** No hay recolección continua ni participativa de datos de ruido a nivel comunal. Los sensores profesionales (sonómetros clase 1) son costosos y de cobertura limitada
3. **Institucional:** Las municipalidades dependen de denuncias para detectar problemas de ruido. La fiscalización de la SMA opera de forma reactiva y con recursos limitados.
4. **Ciudadanía:** Los vecinos no tienen canales accesibles para reportar ruido con datos objetivos. Baja conciencia ciudadana sobre los efectos del ruido en la salud.

Efectos principales:

1. Diagnósticos ambientales incompletos o desactualizados que no reflejan la realidad acústica del territorio.
2. Decisiones de planificación urbana (zonificación, permisos de edificación, transporte) tomadas sin considerar el impacto acústico real.
3. Deterioro de la calidad de vida de los vecinos expuestos a niveles de ruido por sobre la norma, con efectos en salud (estrés, trastornos del sueño, hipertensión).
4. Percepción ciudadana de desamparo frente al ruido, al no contar con datos ni herramientas para visibilizar el problema.

¹ Se adjunta un diagrama Ishikawa en el anexo * que grafica este capítulo.

¿Qué parte ataca 40dB? La solución se enfoca en las causas tecnológicas y de datos: proveer una plataforma que genere datos empíricos de ruido de forma continua y participativa, entregando a la municipalidad un dashboard con información accionable y al vecino un canal para contribuir y visualizar el estado acústico de su entorno.

Descripción del mercado objetivo

Cliente principal: Municipalidades

La municipalidad es quien toma la decisión de adoptar y eventualmente financiar la herramienta. Su necesidad es contar con datos empíricos de ruido para fiscalizar ordenanzas, priorizar intervenciones y reportar a la SMA. Su dolor actual es depender de denuncias puntuales y no tener visibilidad continua del problema.

Perfil del cliente

Quién es: Dirección de Medio Ambiente o Departamento de Fiscalización de una municipalidad.

Necesidad: Monitorear niveles de ruido por zonas, identificar focos críticos, generar reportes para la toma de decisiones.

Escenario de uso: El encargado ambiental accede al dashboard de 40dB, revisa el heatmap de la comuna, identifica zonas con niveles sobre la norma, y prioriza fiscalizaciones o intervenciones.

Usuario final: Vecinos de la comuna

El vecino es quien usa la aplicación para reportar ruido y consultar el estado acústico de su barrio. No paga directamente, pero su participación es esencial para alimentar el sistema de datos.

Perfil del usuario

Quién: Residente de la comuna (cualquier edad, con acceso a un navegador web).

Necesidad: Saber cuánto ruido hay en su zona, poder reportar situaciones de ruido excesivo con datos objetivos.

Escenario de uso: El vecino abre la app web en su celular, el micrófono captura automáticamente el nivel de dB por unos segundos, el usuario selecciona el tipo de fuente de ruido (tráfico, construcción, local nocturno, etc.) y envía el reporte geolocalizado.

Modelo de relación

El modelo es **B2G** (Business to Government): 40dB se ofrece a la municipalidad como herramienta de gestión, y la municipalidad habilita el acceso público para sus vecinos. El vecino es el usuario que genera datos; la municipalidad es el cliente que consume la información procesada.

Objetivo general y objetivos específicos

Objetivo general

Desarrollar una plataforma web de monitoreo colaborativo de ruido urbano que permita a la Municipalidad de Maipú visualizar en tiempo real los niveles de contaminación acústica por zonas, combinando reportes ciudadanos con datos de sensores IoT, para facilitar la toma de decisiones en gestión ambiental.

Objetivos específicos

1. **Diseñar e implementar** una aplicación web que permita a los vecinos reportar niveles de ruido desde su navegador, capturando dB mediante el micrófono del dispositivo junto con la geolocalización y el tipo de fuente sonora.
2. **Desarrollar** un backend con API REST que reciba, almacene y procese los datos de reportes ciudadanos y lecturas de sensores IoT, y los exponga para su visualización en un mapa de calor interactivo.
3. **Construir** un prototipo funcional de sensor IoT, potencialmente con ESP32 y un sensor de sonido, capaz de enviar lecturas periódicas de niveles de ruido al backend, y diseñar el mecanismo de cruce de datos entre sensores fijos y reportes ciudadanos para establecer niveles de confiabilidad.
4. **Implementar** un dashboard municipal que permita visualizar el heatmap de ruido, filtrar por zona, horario y tipo de fuente, y generar reportes exportables con los datos recopilados.

Situación inicial del cliente (AS-IS)²

Proceso actual de gestión del ruido en una municipalidad

Hoy, el flujo de gestión del ruido en una municipalidad chilena típica funciona de la siguiente manera:

1. **Un vecino percibe ruido molesto** en su barrio (construcción, local nocturno, tráfico, etc.).
2. **El vecino presenta una denuncia** a través de los canales disponibles de la municipalidad (fono municipal, correo electrónico, presencial) o directamente ante la SMA.
3. **La municipalidad asigna un inspector** que se desplaza al lugar con un sonómetro portátil para realizar una medición puntual.
4. **El inspector mide en terreno**, compara con los límites de la ordenanza municipal y/o el DS 38, y elabora un informe.
5. **Si se detecta infracción**, se inicia un procedimiento sancionatorio ante el Juzgado de Policía Local o se deriva a la SMA.
6. **No existe registro centralizado** ni georreferenciado de las mediciones ni de los niveles de ruido históricos por zona. La información queda dispersa en informes individuales.

Puntos de dolor

Reactivo: Solo se actúa cuando hay denuncia; no hay monitoreo preventivo.

Puntual: La medición es de un momento y lugar específico; no refleja patrones temporales ni espaciales.

Sin datos históricos accesibles: No hay forma de comparar niveles de ruido entre zonas ni de ver tendencias en el tiempo.

Costoso en recursos humanos: Cada denuncia requiere desplazamiento de un inspector con equipamiento.

Opaco para el vecino: El denunciante no tiene acceso a datos ni seguimiento del estado acústico de su entorno.

² Se adjunta un diagrama AS-IS que grafica este capítulo. Se encuentra en el anexo “Diagrama AS IS.png” adjunto en el envío.

Planificación

El proyecto se desarrollará siguiendo una metodología ágil adaptada de Scrum, organizada en 6 sprints de 2 semanas a lo largo de 12 semanas especificados en una Carta Gantt³. Cada sprint termina con un entregable funcional que permite validar avances de forma incremental, obtener una retroalimentación temprana y ajustar el alcance según lo aprendido.

Sprint	Semanas	Objetivo	Entregable Funcional
S0 — Kickoff	1 - 2	Fundamentos técnicos: definir arquitectura, levantar requerimientos, crear repositorios y desplegar entornos base en la nube.	Frontend y backend desplegados y conectados con un endpoint dummy; base de datos creada con modelo inicial.
S1 — Reporte	3 - 4	Flujo completo mínimo: autenticación + envío de un reporte manual + visualización en el mapa.	Un vecino autenticado puede enviar un reporte geolocalizado y verlo como punto en el mapa.
S2 — Audio + heatmap	5 - 6	Captura de dB reales desde el navegador y visualización como mapa de calor.	El reporte mide automáticamente el nivel de dB; el mapa muestra un heatmap en lugar de pines individuales.
S3 — IoT integrado	7 - 8	Integración del prototipo de sensor físico con el backend.	Prototipo ESP32 + Sensor de audio enviando datos reales vía HiveMQ, visibles en el heatmap junto a los reportes ciudadanos.
S4 — Cruce + dashboard	9 - 10	Algoritmo de confiabilidad y panel para la municipalidad.	Dashboard municipal con filtros (zona, horario, tipo), métricas agregadas y reportes validados contra sensores con índice de

³ Revisar Anexo “Carta Gantt.xlsx” adjuntado en el envío del proyecto.

			confiabilidad.
S5 — Refinamiento y entrega	11 - 12	Pulido, exportación, documentación y presentación final.	Producto terminado desplegado, documentación técnica y demo lista.

Servicios Cloud a utilizar (IaaS / PaaS / SaaS)

La solución se desplegará en la nube para garantizar disponibilidad, escalabilidad y acceso remoto tanto para los sensores IoT como para los usuarios de la plataforma. Se utilizará Render, Vercel y Supabase, aprovechando sus capas gratuitas (Free Tier) dado que el presupuesto es simulado/académico.

Componente	Servicio	Tipo Cloud	Justificación
Servidor de aplicación (Backend FastAPI)	Render	PaaS	Render abstrae la infraestructura: despliegue directo desde Git, SSL automático, sin administrar servidor.
Base de datos	Supabase (PostgreSQL con PostGIS)	PaaS	Base de datos gestionada con backups automáticos, escalado y soporte nativo para extensiones geoespaciales. Libera al equipo de administrar el motor de BD.
Almacenamiento	Supabase Bucket	IaaS	Almacenamiento de objetos para exportaciones de reportes, respaldos de datos y archivos estáticos del frontend.
Frontend (Vue3)	Vercel	PaaS	Despliegue automático desde Git con CDN global, integración nativa con frameworks JS.

Recepción de datos IoT	HiveMQ	PaaS	Servicio gestionado para recibir mensajes MQTT desde los sensores ESP32. Maneja autenticación de dispositivos, enrutamiento de mensajes y escalado automático.
Autenticación	Supabase Auth	SaaS	Servicio de identidad gestionado por Supabase. Permite delegar a esta plataforma el proceso de autenticación y autorización de los usuarios.

Conceptualización (solución propuesta – versión preliminar)⁴

Descripción general (TO-BE)

40dB es una plataforma web de monitoreo colaborativo de ruido urbano que opera bajo un modelo híbrido de recolección de datos: reportes ciudadanos desde el navegador y lecturas automáticas de sensores IoT fijos. Los datos se procesan y visualizan en un mapa de calor interactivo accesible tanto para vecinos como para el equipo técnico municipal.

El flujo con 40dB implementado sería:

1. **Vecinos** abren la app web, el micrófono del dispositivo captura el nivel de dB durante unos segundos, el usuario categoriza el tipo de ruido y envía el reporte geolocalizado.
2. **Sensores IoT** instalados en puntos estratégicos envían lecturas de dB cada 5 minutos al backend vía MQTT.
3. **El backend** recibe ambos flujos, los almacena con coordenadas y timestamp, y ejecuta el cruce de datos: los reportes ciudadanos que caen dentro de un radio establecido de un sensor fijo se comparan con la lectura del sensor en el mismo rango horario, asignándoles un índice de confiabilidad.
4. **El heatmap** se genera interpolando todos los puntos de datos (sensores + reportes ponderados por confiabilidad) y se visualiza sobre un mapa de la comuna.
5. **El dashboard municipal** permite filtrar por zona, rango horario, tipo de fuente y período, y exportar reportes en PDF/CSV.

Funcionalidades principales

1. **Reporte ciudadano con captura de dB:** Interfaz web que utiliza la Web Audio API para medir el nivel de presión sonora desde el micrófono del dispositivo, combinado con geolocalización (GPS del navegador) y selección del tipo de fuente de ruido.
2. **Mapa de calor interactivo (heatmap):** Visualización en Leaflet.js sobre OpenStreetMap de los niveles de ruido por zona, actualizado en tiempo real con los datos entrantes. Accesible para vecinos y municipalidad.

⁴ Revisar diagrama de arquitectura en el anexo “Diagrama Arquitectura.png” adjuntado en el envío.

3. **Ingesta de datos IoT:** Endpoint MQTT/HTTP para recibir lecturas de sensores ESP32 con nivel de dB, timestamp y coordenadas fijas registradas en el sistema.

4. **Motor de cruce y confiabilidad:** Algoritmo que compara reportes ciudadanos con lecturas de sensores cercanos para asignar un puntaje de confiabilidad a cada reporte, ponderando su peso en el heatmap.

5. **Dashboard municipal:** Panel con filtros avanzados (zona, hora, fecha, tipo de fuente), estadísticas agregadas (promedio dB por zona, horarios críticos, evolución temporal) y exportación de reportes.

6. **Historial y tendencias:** Almacenamiento de datos históricos que permite visualizar la evolución del ruido por zona en el tiempo, comparar períodos y detectar patrones.

Consideraciones de calidad y seguridad

Privacidad: Los reportes ciudadanos son anónimos. No se almacena audio, solo indicadores acústicos (dB) y metadatos.

Validación de datos: Los reportes pasan por validación básica (rango de dB plausible, geolocalización dentro de la comuna, rate limiting por IP/dispositivo para evitar spam).

Seguridad IoT: Los sensores se autentican ante HiveMQ mediante certificados, evitando la inyección de datos falsos.

Respaldo: La base de datos en PostgreSQL gestionada en Supabase cuenta con backups automáticos diarios.

Autenticación: Tanto los funcionarios municipales como los vecinos que envían reportes deben iniciar sesión con una cuenta de con su correo y contraseña. El backend valida el token con Supabase. Este enfoque cumple un doble propósito: evita gestionar directamente las cuentas, sino que mediante Supabase Auth y aprovecha la seguridad de Supabase y previene el spam y los reportes falsos al asociar cada reporte a una identidad verificada, permitiendo además aplicar rate limiting por usuario y detectar comportamientos anómalos. Los roles se diferencian en el backend: los vecinos solo pueden crear reportes y visualizar el mapa público,

mientras que los funcionarios municipales acceden al dashboard completo con métricas y exportación de datos.

Alcance, supuestos y restricciones

Alcance (incluye)

- Aplicación web funcional para reporte ciudadano con captura de dB y geolocalización.
- Backend con API REST para ingesta de reportes y datos de sensores.
- Mapa de calor interactivo sobre la comuna de Maipú.
- Prototipo de al menos 1 sensor IoT (ESP32 + sensor de audio) enviando datos al backend.
- Dashboard municipal con filtros básicos y exportación de datos.
- Mecanismo de cruce de datos sensor/reporte con índice de confiabilidad.
- El sensor cuando este encendido debe enviar información al broker cada 10 segundos.

No alcance (excluye)

- Aplicación móvil nativa (iOS/Android). La solución es web responsive.
- Despliegue de una red extensa de sensores. Se prototipa con 1–2 unidades.
- Integración directa con sistemas de la SMA o del MMA.
- Calibración profesional certificada de las mediciones (el proyecto genera datos indicativos, no mediciones normativas con validez legal).
- Gestión de denuncias formales ni flujos sancionatorios.

Entregables

- Código fuente del frontend, backend y firmware del sensor.
- Base de datos poblada con datos de prueba y datos reales del piloto.
- Documentación técnica (arquitectura, API, modelo de datos).
- Informe final del proyecto.
- Presentación de defensa del prototipo.

Supuestos

- Los navegadores modernos permiten acceso al micrófono del dispositivo vía Web Audio API con el consentimiento del usuario.
- Se dispondrá de al menos un punto con acceso a WiFi y electricidad para instalar el sensor piloto en Maipú.
- Los usuarios del piloto contarán con dispositivos con GPS y micrófono funcional.
- HiveMQ Free Tier será suficiente para el volumen de datos del prototipo.

Restricciones

- **Tiempo:** 12 semanas de desarrollo.
- **Equipo:** 2 integrantes.
- **Presupuesto:** Simulado/académico. Hardware IoT con costo real estimado bajo (10.000 -- 15.000 CLP por sensor).
- **Precisión de mediciones:** Los micrófonos de smartphones no están calibrados profesionalmente; los datos son indicativos y comparativos, no normativos.
- **Cobertura:** El piloto se limita a la comuna de Maipú, con datos representativos, pero no exhaustivos.

Metodología de desarrollo y gestión del proyecto

El desarrollo de 40dB se gestiona con una metodología ágil adaptada de scrum, coherente con la planificación presentada previamente: La idea de 6 sprints de 2 semanas a lo largo de 12 semanas, cada uno cerrado con una entrega funcional. Scrum se eligió porque el proyecto tiene requerimientos que maduran a medida que se construye el prototipo y un enfoque iterativo permite entregar valor de forma incremental, validar supuestos temprano y ajustar el alcance sin comprometer la fecha de cierre. Dado que el equipo es de solo 2 integrantes, se aplica una variante liviana: no existe un scrum master dedicado y, dentro de cada sprint, el trabajo se organiza con un flujo tipo kanban sobre un tablero de tareas.

Trabajo semana a semana: planificación, seguimiento y control

- **Planificación del sprint:** Al inicio de cada sprint de 2 semanas, el equipo selecciona elementos del backlog de producto, los descompone en tareas y estima su esfuerzo. El objetivo del sprint queda definido por el entregable funcional comprometido.
- **Seguimiento:** Las tareas se gestionan en un tablero de **Trello** con columnas Backlog, “nombrarlas”. Al ser un equipo de 2 personas, las reunión diaria se reemplazan por *sincronizaciones asíncronas* breves y al menos tres reuniones de coordinación presenciales o remotas por semana
- **Control de versiones:** El código se gestiona en repositorios Git con ramas principales main y dev. Todo cambio se integra mediante Pull Request revisado por el otro integrante antes de hacer merge a dev.
- **Cierre del sprint:** Cada sprint termina con una revisión (demostración del entregable funcional) y una retrospectiva en la que se identifica que funcionó, que no, y que ajustar para el siguiente ciclo.

Roles y responsabilidades del equipo

El equipo está compuesto por 2 integrantes, ambos en el rol de desarrollador. La distribución de responsabilidades principales, sin perjuicio de que las decisiones de arquitectura, revisión de código, pruebas y documentación se asumen de forma conjunta, es la siguiente:

Área de responsabilidad	Responsabilidad principal	Alcance
Backend, base de datos e infraestructura cloud	Joaquín Meléndez	API REST en FastAPI, modelo de datos PostGIS, despliegue en Render y Supabase.

Frontend y subsistema IoT	Ignacio Saavedra	Aplicación web en Vue 3, mapa de calor, firmware del sensor ESP32 e ingesta MQTT.
Responsabilidades compartidas	Ambos integrantes	Definición de arquitectura, revisión de Pull Requests, pruebas, documentación técnica y preparación de la presentación.

La asignación corresponde a la responsabilidad principal de cada integrante; ambos pueden colaborar transversalmente cuando un sprint lo requiera.

Definición de Hecho (Definition of Done) y criterios de calidad

Una tarea o funcionalidad se considera hecha cuando se cumple todas las siguientes condiciones:

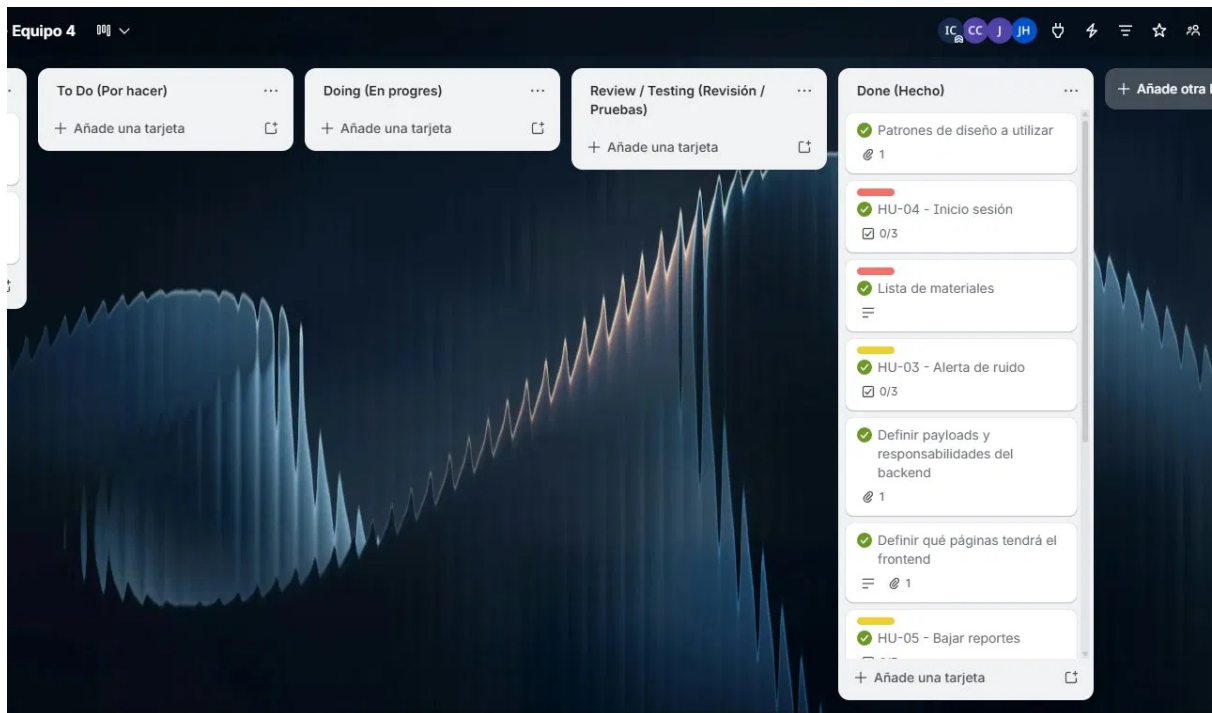
- El código implementa los criterios de aceptación de la historia de usuario correspondiente.
- Fue revisada y aprobada en un Pull Request por el otro integrante.
- Incluye pruebas mínimas cuando se aplica, o queda registrada como deuda técnica si la cobertura se posterga.
- Está integrada en la rama dev y desplegada en el entorno correspondiente.

Como criterios de calidad mínimos transversales se exigen: validación de entradas, manejo explícito de errores, control de acceso por rol, ausencia de secretos en el repositorio.

Hitos del proyecto: los hitos corresponden a los entregables funcionales de cada sprint (S0 a S5) descritos en el capítulo de planificación, cuyas fechas detalladas se encuentran en el anexo “carta gantt”⁵

(<https://docs.google.com/spreadsheets/d/1jX1qrzsDmBgooaQ3M7lFDdhv6EdEumLV/edit?usp=sharing&ouid=107429315056806719482&rtpof=true&sd=true>)

⁵ Revisar Anexo “Carta Gantt.xlsx” y “Tablero Trello.pdf” adjuntado en el envío del proyecto.



Ejemplo de Trello

Documentos y diagramas de diseño de la solución

Para guiar el desarrollo, el equipo elaboró un conjunto de documentos y diagramas de diseño. A continuación, se describe, para cada uno, su propósito, que representa y cómo se relaciona con el problema y los objetivos del proyecto. Los diagramas se adjuntan como anexos de esta entrega (Adjuntados en el zip de Diagramas y carta gantt.zip)

1. Diagrama de flujo del usuario (User Flow) y BPMN del proceso

- a. **Propósito:** representar paso a paso la interacción de los usuarios con el sistema
- b. **Que representa:** por un lado, el diagrama AS-IS, que modela el proceso actual de gestión del ruido en la municipalidad; por otro, los flujos TO-BE: el vecino genera un reporte, el vecino consulta el heatmap, el funcionario que gestiona una alerta y el flujo de inicio de sesión.
- c. **Relación con el problema y objetivos:** El diagrama AS-IS evidencia los puntos de dolor del proceso reactivo actual, mientras que los flujos TO-BE muestran como 40dB los resuelve, sirviendo de guía directa para implementar las vistas y la navegación de la aplicación.

2. Mockups de las pantallas principales

- a. **Propósito:** definir la estructura y disposición de la interfaz antes de programarla.
- b. **Que representa:** las pantallas del producto, la landing pública, el inicio de sesión, el home del vecino, la bandeja del funcionario con el modal del cambio de estado, el home de administrador y la vista de heatmap con filtros y leyenda.
- c. **Relación con el problema y objetivos:** los wireframes son la base del sistema de diseño (tokens y componentes base reutilizable) y se materializan en las vistas implementadas, aseguran una experiencia coherente y accesible para distintos públicos, desde el vecino hasta el funcionario.

3. Modelo de datos

- a. **Propósito:** definirla estructura de persistencia de la información.
- b. **Que representa:** las entidades **comuna**, **tipo_estado**, **usuario**, **sensor**, **lectura**, **reporte** e **historial_estado**, con sus relaciones principales: un usuario extiende 1:1 la identidad de autenticación, un sensor genera muchas lecturas (1:N), un reporte puede vincularse a una lectura como evidencia, y un reporte acumula un historial de estados. Las entidades geográficas incorporan una columna de ubicación con PostGIS.

- c. **Relación con el problema y objetivos:** este modelo habilita el cruce geo-temporal entre reportes ciudadanos y lecturas de sensores, que es el núcleo diferenciador de la solución.

4. Diagrama de arquitectura de alto nivel

- a. **Propósito:** Mostrar los componentes del sistema y como se comunican entre sí.
- b. **Que representa:** el frontend (aplicación web Vue desplegada en Vercel), el backend (API FastAPI en Render), la base de datos (PostgreSQL con PostGIS en Supabase), el broker MQTT (HiveMQ Cloud), los sensores ESP32 y el servicio de autenticación.
- c. **Relación con el problema y objetivos:** evidencia como el modelo híbrido de datos (reportes más telemetría IoT) se realiza técnicamente y como cada componente contribuye al objetivo de entregar un mapa de ruido.

5. Diagrama de casos de uso e historias de usuario

- a. **Propósito:** delimitar el alcance funcional del sistema identificando actores y las acciones que cada uno puede realizar.
- b. **Que representa:** los actores **Vecino(ciudadano)**, **Funcionario municipal** y **Administrador**, junto con las historias que estructuran el desarrollo.
- c. **Relación con el problema y objetivos:** cada historia de usuario se vincula directamente con un objetivo específico; el reporte ciudadano, el dashboard municipal y la visualización del heatmap y asegura que el desarrollo cubra todos los actores del modelo B2G.⁶

⁶ Revisar los flujos TO-BE en los anexos "Carpeta Flujos TO-BE".
Mockup inicial en anexo "40dB — Prototipo Frontend.pdf".
Modelo de datos en anexo "Modelo de datos.pdf".
Diagrama arquitectura en anexo "Diagrama C4 - 40dB.png"
El proceso AS-IS se encuentra en el anexo "Diagrama AS IS.png".

Arquitectura de software y patrones de diseño

Arquitectura general

40db adopta una arquitectura **cliente-servidor multicapa**. La capa de presentación es una aplicación web construida en Vue 3; la capa de aplicación y lógica de negocio es un backend en FastAPI; y la capa de datos es una base de PostgreSQL con la extensión geoespacial PostGIS. A estas se suma un canal sincrónico de ingesta IoT sobre el protocolo MQTT, que alimenta el sistema con la telemetría de los sensores.

El desafío central de la arquitectura es la asimetría de tráfico: el frontend genera peticiones HTTP esporádicas, mientras que los sensores producen un flujo MQTT constante. La solución es un proceso único con doble punto de entrada: un solo proceso FastAPI atiende las peticiones HTTP y, mediante el ciclo de vida de la aplicación, levanta de forma asíncrona un suscriptor MQTT.

Arquitectura interna del backend: hexagonal lite

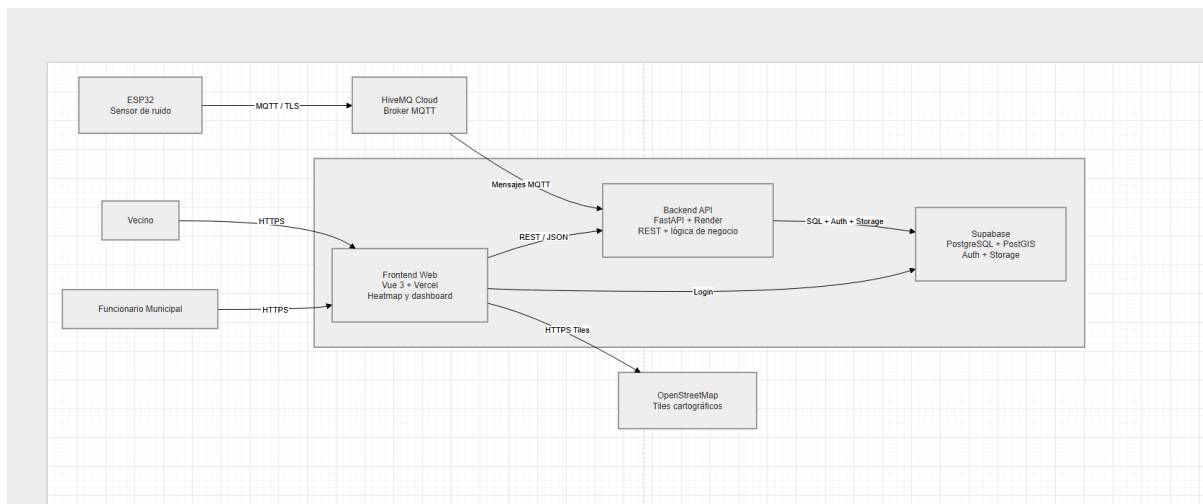
El backend sigue en **arquitectura hexagonal (Puertos y Adaptadores) en versión liviana**. Los puertos y adaptadores se aplican únicamente en los bordes de entrada/salida, el acceso a Supabase, la mensajería MQTT y la API HTTP, mientras que el núcleo es una orquestación delgada, dado que gran parte de la lógica de negocio (el cruce geo-temporal) reside en funciones de PostgreSQL. La regla de dependencia es estricta: siempre apuntan hacia dentro. Las capas api y mqtt dependen de la capa de aplicación (casos de uso), que depende de los puertos del dominio, que no depende de nada. La infraestructura implementa esos puertos. Esto mantiene el dominio testeable y desacoplado de la tecnología concreta.

Se implementó una variante “lite” porque una arquitectura hexagonal completa, con inyección de dependencia externa, sería sobre ingeniería para un prototipo desarrollado por 2 personas en 12 semanas; el mecanismo de dependencias propio de FastAPI es suficiente.

Arquitectura interna del frontend

⁷El frontend usa una arquitectura **basada en componentes** con una estratificación de **vistas finas, compostables y servicios**. Las vistas orquestan y muestran, los compostables encapsulan lógica reactiva reutilizable (por ejemplo, la carga de datos del heatmap con auto-refresco).

⁷ Se ocupa el concepto de hexagonal lite haciendo referencia al concepto adaptado de clean code pero contextualizado a nuestro proyecto ya que no requeríamos de arquitectura completa



Tecnologías, lenguaje y dependencias (stack definitivo)

A continuación, se presenta el stack tecnológico definitivo del proyecto. Respecto de la versión preliminar de servicios cloud, la persistencia y la autenticación se consolidaron en **Supabase**, que provee PostgreSQL gestionando con PostGIS y un servicio de autenticación integrado, reduciendo la carga operativa del equipo.

Frontend

- **Lenguaje y framework:** Typescript con **Vue 3**, empaqueta con **Vite**.
- **Librerías clave:** Pinia (gestion de estado), Vue Router en modo history (Enrutamiento), leaflet más leaflet heat (Mapa de calor), jsPDF, jspdf-autotable y html2canvas (exportacion de reportes).
- **Calidad:** ESLint y oxlint mas prettier (linting y formato); Vitest con @vue/test-utils (pruebas unitarias) y Cypress (pruebas end-to-end).

Backend

- **Lenguaje y framework:** Python con FastAPI servido por uvicorn
- **Librerías clave:** supabase, paho-mqtt (cliente MQTT del ingestor), pydantic-settings (configuración por variables de entorno), pyjwt (verificación local de JWT emitidos por Supabase Auth), python-ulid (generación de correlation ids para trazabilidad de request).
- **Base de datos:** PostgreSQL con la extensión de PostGIS.
- **Mensajería:** HiveMQ Cloud como broker MQTT.

Subsistema IoT

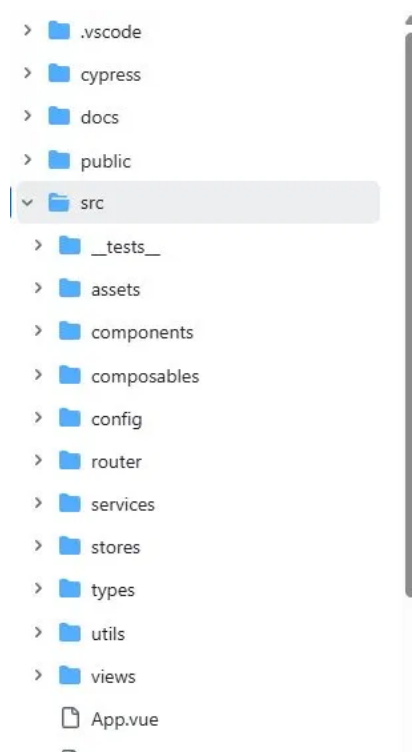
- **Hardware:** microcontrolador ESP32 con un sensor de sonido.
- **Firmware:** desarrollado en C/C++ (Entorno Arduino), publicando lecturas vía MQTT sobre TLS.

Estándares

- **Estructura de carpetas:** el backend sigue la estructura hexagonal lite (dominio, aplicacion, infraestructura y api); el frontend se organiza por responsabilidad (Componentes comunes, de layout y de mapa; componables; servicios; stores; tipos; y vistas por rol), con el alias @ apuntando a src/.
- **Linters y formato:** ESLint, oxlint y Prettier en el frontend; estilo PEP 8 en el backend. La API expone sus endpoints bajo el prefijo /api/v1.
- **Convención de commits:** mensajes en minúsculas y en español, con prefijos feat:, fix:, chore: y add:.

Repositorios y flujo de trabajo

El proyecto se gestiona en dos repositorios Git (frontend y backend). Cada repositorio mantiene las ramas principales **main** (estable) y **dev** (integración). El flujo de trabajo es: desarrollo en una rama de funcionalidad, apertura de Pull Request, revisión por el otro integrante, merge a dev y, integración a main.



Estructura de frontend.

☐	 Sync dev → main: landing, dashboard admin, home vecino, home funciona... ✓ #20 by JoaquinMelendez (Collaborator) was merged 4 days ago
☐	 Sync dev → main: landing, dashboard admin, home vecino, home funciona... ✓ #19 by JoaquinMelendez (Collaborator) was merged 4 days ago
☐	 add: initial CLAUDE.md to persist the directions to backend documenta... ✓ #18 by JoaquinMelendez (Collaborator) was merged 4 days ago • Approved
☐	 Sync dev → main: landing, dashboard admin, home vecino, home funcionario, docs HU ✓ #17 by delll000 (Owner) was merged 4 days ago • Approved
☐	 docs: diagramas de flujo de usuario por rol #16 by delll000 (Owner) was merged 2 weeks ago • Approved
☐	 feat: home del funcionario con gestion de alertas HU-002 #15 by delll000 (Owner) was merged 5 days ago • Approved
☐	 feat: home del vecino con heatmap embebido y formulario de reporte #14 by delll000 (Owner) was merged 5 days ago • Approved

Ejemplo de PR's y commit

Configuración de servidor de producción (paso a paso)

Plataforma y componentes

- **Backend (FastAPI):** desplegado en **Render** (PaaS), con SSL y dominio gestionados automáticamente.
- **Base de datos y autenticación: Supabase**, que provee PostgreSQL gestionado con la extensión PostGIS y el servicio Supabase Auth.
- **Frontend (Vue/Vite):** desplegado en **Vercel** (PaaS) con CDN global.
- **Broker MQTT: HiveMQ Cloud**, en su capa gratuita, con TLS obligatoria (puerto 8883)
- **Almacenamiento de objetos:**

Requisitos del servidor

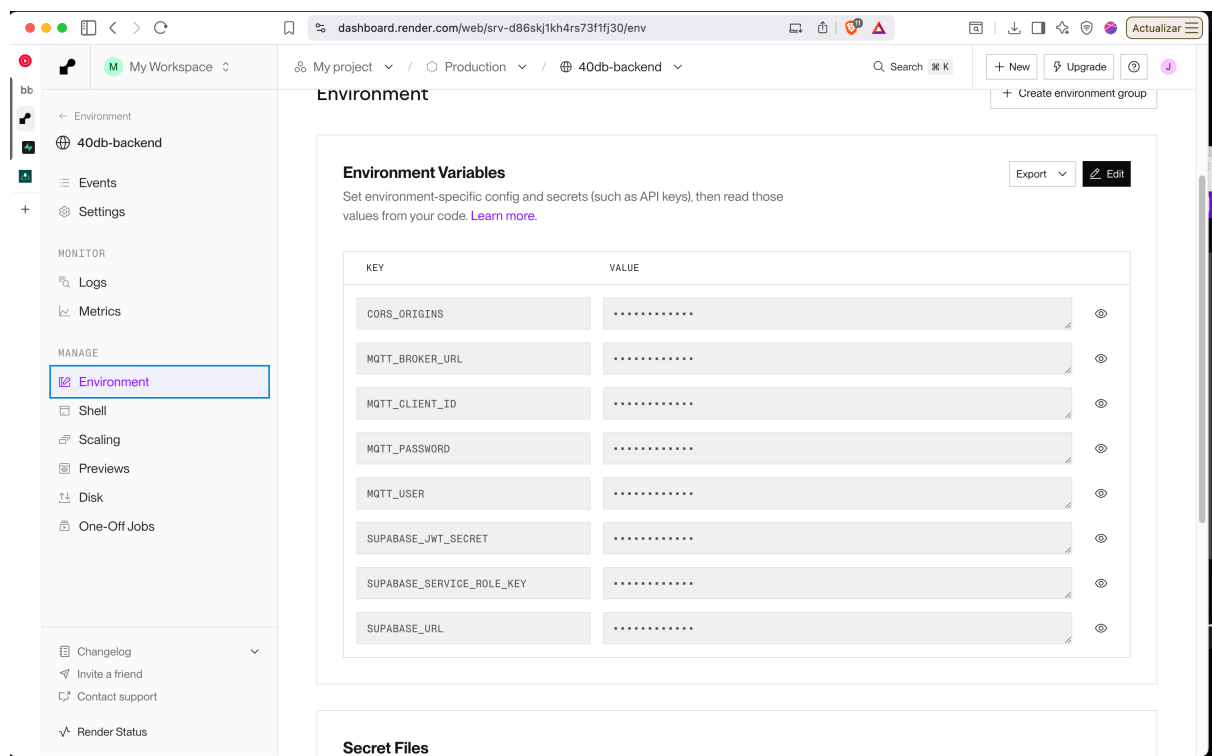
- **Runtime:** Python para el back, Node 20+ para el build del frontend (Vite)
- **Variables de entorno del backend:**
 - **Para Supabase / Frontend**
 - SUPABASE_URL
 - SUPABASE_SERVICE_ROLE_KEY
 - SUPABASE_JWT_SECRET
 - CORS_ORIGINS
 - **MQTT**
 - MQTT_BROKER_URL
 - MQTT_USER
 - MQTT_PASSWORD
 - MQTT_CLIENT_ID
- **Variables del entorno del frontend:**
 - VITE_API_URL
 - anon_key (supabase)
- **Puertos:**
 - Backend expone 8000 (Render publica vía 443/SSL); broker MQTT en 8883 (TLS).
- **Permisos:**
 - service_role_key

Instrucciones paso a paso

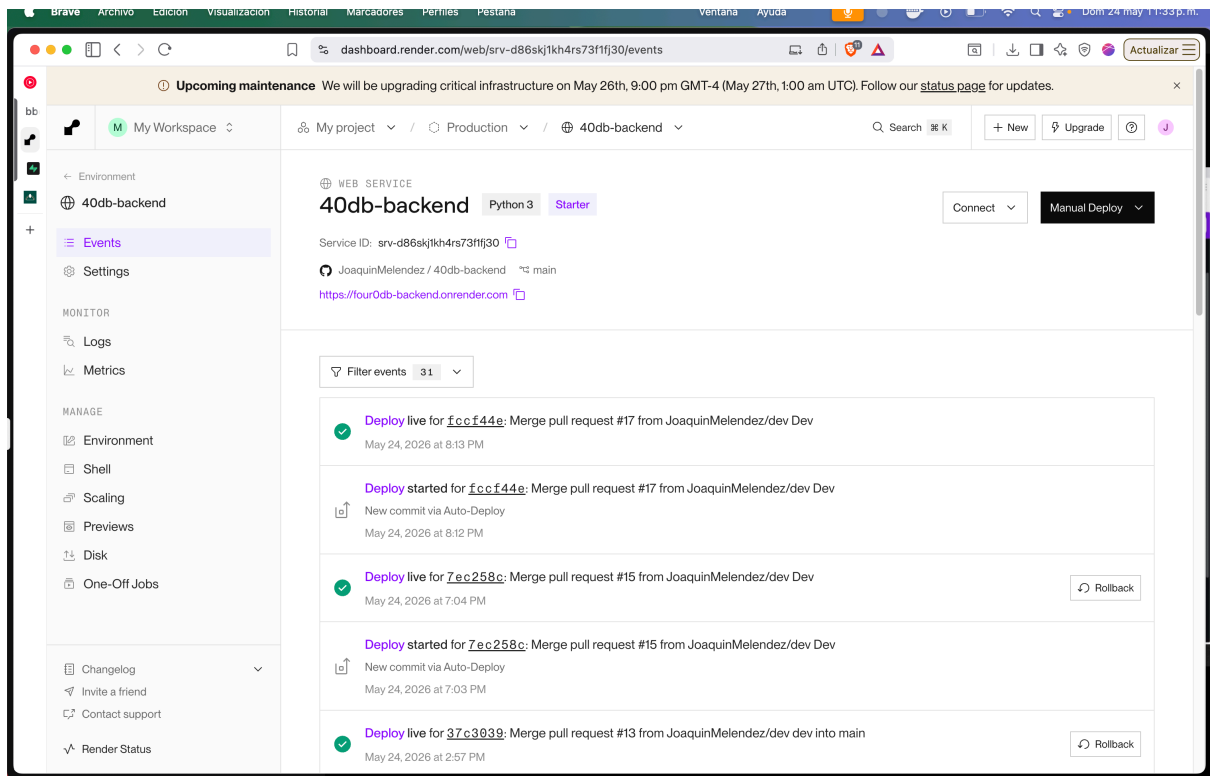
- **Provisión de DB:** crear proyecto Supabase, habilitar PostGIS, correr migraciones (supabase db reset local → supabase db push).

- Instalación de lenguaje y dependencias: `python -m venv .venv`, `pip install -r requirements.txt`.
- Configuración de secretos: cargar variables en el panel de Render (backend) y Vercel (frontend).
- Despliegue: push a main dispara el build automático en Render y Vercel.
- Configuración HiveMQ: crear cluster gratuito, usuario backend-subscriber (SUBSCRIBE 40db/#) y usuario 40db-sensor (PUBLISH 40db/sensores/+lectura), descargar cert CA (ver [iot.md §18](#)).
- Logs y monitoreo: logs nativos de Render para el backend; `/health/live` y `/health/ready` para el orquestador (ver [api.md API2](#) / [backend.md ADR 06](#)).

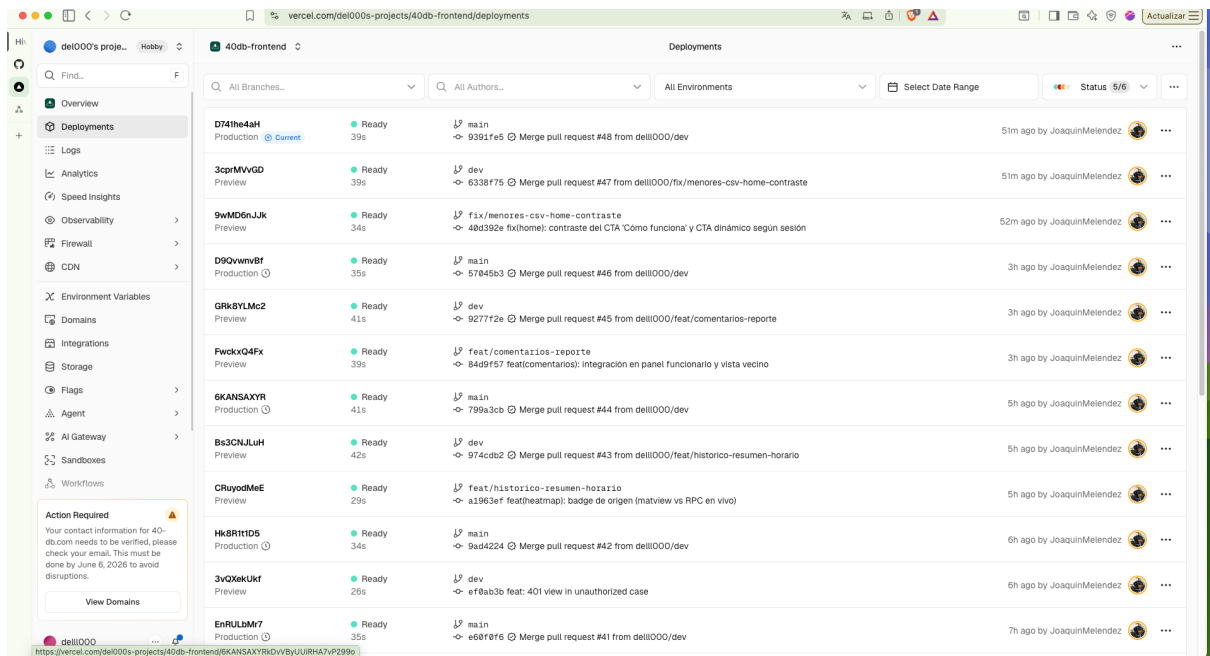
Evidencias:



- <https://four0db-backend.onrender.com>



Frontend:



Estado de avance del desarrollo (evidencia de producto funcional)

El desarrollo del prototipo se encuentra en un estado avanzado: el sistema **opera de extremo a extremo con datos reales**. Un sensor físico basado en **ESP32** captura niveles de ruido y publica sus lecturas mediante el broker **MQTT**; el **backend** las ingiere y las persiste en la base de datos **Supabase/PostGIS**; y el **frontend** consume esa información a través de la API REST y la visualiza en el mapa de calor y en las vistas por rol. Actualmente hay un único sensor ESP32, por lo que el heatmap refleja las lecturas de ese dispositivo; arquitectura admite N sensores y la incorporación de más unidades es una tarea de despliegue de hardware, no de desarrollo de software. La siguiente tabla detalla las funcionalidades y su estado.

Funcionalidad	Estado	Descripción
1. Sistemas de diseño	Implementada	Tokens de diseño en variables CSS y componentes base reutilizables en el frontend
2. Routing con control de acceso por rol	Implementada	Rutas con carga diferida, guards de autenticación y rol, y restauración de la sesión antes de montar el router.
3. Mapa de calor interactivo (heatmap)	Implementada	Render con Leaflet y leaflet heat sobre OpenStreetMap, centrado en Maipú; filtros reactivos, auto-refresco, leyenda de bandas dB y banner de alerta sostenida. Consume las lecturas reales servidas por el backend
4. Exportación de reportes del heatmap	Implementada	Generación 100% en cliente en formatos PNG, CSV y PDF, este último con encabezado institucional y tabla de indicadores.
5. Vistas funcionales por rol	Implementada	Landing pública, inicio de sesión, home del vecino con modal de reporte, bandeja del funcionario con gestión de alertas y auditoría, y home del administrador.

		Operan sobre datos reales provenientes de la API.
6. Capa de servicios con contratos asíncronos	Implementada	Servicios (heatmap, sensores, alertas, indicadores y reportes) con firma asíncrona, listos para reemplazo directo por llamadas HTTP.
7. Diseño y documentación del backend	Implementada	Arquitectura, modelo de datos PostGIS, contrato HTTP con endpoints y subsistema IoT especificados en la documentación técnica.
8. API REST del backend (FastAPI)	Implementada	Endpoints REST sobre arquitectura hexagonal lite operativos; sirven datos al frontend y reciben la telemetría del subsistema IoT.
9. SQL definitivo	Implementada	Esquema PostGIS aplicado en Supabase: tablas, índices GIST, triggers y funciones RPC en uso del sistema.
10. Integración frontend-backend y autenticación	Implementada	El front consume la API real: autenticación por correo y contraseña via Supabase Auth y manejo uniforme de errores en funcionamiento.
11. Subsistema IoT (firmware e ingestor MQTT)	Implementada	Firmware del ESP32 publicando lecturas de dB via MQTT e ingestor del backend suscrito e insertando en la base de datos. Hay un sensor físico; la solución escala a más unidades sin cambios de código.
12. Cobertura de pruebas automatizadas	En progreso	Vitest, Cypress y pytest configurados; ampliando la cobertura más allá de los specs de ejemplo.

Integraciones necesarias

El funcionamiento de 40dB depende de varias integraciones con servicios externos. A continuación, se describe cada una, indicando que datos entran y salen y las consideraciones de seguridad aplicables.

Autenticación (Supabase Auth)

- **Que hace:** gestiona el registro y el inicio de sesión de los usuarios.
- **Datos:** el frontend envía las credenciales a Supabase Auth y recibe un token JWT, el backend verifica ese token de forma local, sin realizar un viaje y vuelta al servicio.
- **Decisión de alcance:** el MVP utiliza autenticación por correo electrónico y contraseña a través de Supabase auth.
- **Seguridad:** el frontend usa la anon_key solo para registro e inicio de sesión, la service_role_key se mantiene exclusivamente en el backend, el token JWT se verifica con un secreto compartido.

Base de datos gestionada (Supabase / PostgreSQL con PostGIS)

- **Que hace:** persiste usuarios, sensores, lecturas y reportes e historial de estados.
- **Datos de entrada:** el backend, único cliente de la base de datos, lee y escribe usando la service_role_key.
- **Seguridad:** todas las tablas tiene RLS habilitada, el frontend nunca accede directamente a las tablas de negocio.

Broker MQTT (HiveMQ Cloud)

- **Que hace:** provee el canal de telemetría IoT.
- **Datos de entrada:** los sensores ESP32 publican lecturas de dB, en mensaje JSON con el nivel medido y su timestamp en un topic por sensor, el backend esta suscrito a estos topics e inserta las lectura.
- **Seguridad:** TLS obligatorio en el puerto 8883 y credenciales por dispositivo.
- **Límites:** se utiliza la capa gratuita de HiveMQ, suficiente para el volumen del prototipo.

Cartografía base (OpenStreetMap via Leaflet)

- **Que hace:** provee las “tiles” del mapa sobre el que se dibuja el heatmap.
- **Datos de salida:** el cliente solicita las imágenes de las teselas al servidor de OpenStreetMap.
- **Consideración:** se mantiene la atribución correcta de OpenStreetMap y se respeta su política de uso de “tiles”.

Consideraciones transversales y pruebas

De forma transversal, todos los secretos se gestionan mediante variables de entorno y nunca se versionan, todos los canales usan TLS, y la API aplica CORS.

Conclusión

La contaminación acústica es un problema creciente en las ciudades chilenas que carece de herramientas tecnológicas accesibles para su monitoreo continuo a nivel comunal. Las soluciones existentes —mapas de ruido modelados por software y registros de denuncias— son valiosas pero insuficientes para una gestión ambiental proactiva, ya que entregan diagnósticos estáticos o reactivos que no reflejan la realidad acústica en tiempo real ni permiten priorizar intervenciones con base en datos empíricos.

40dB aborda esta brecha mediante una plataforma web que combina la participación ciudadana con sensores IoT para generar datos acústicos continuos y geolocalizados. El mecanismo de cruce entre ambas fuentes —donde las lecturas de los sensores actúan como ground truth para validar los reportes de los vecinos— aporta un enfoque novedoso que balancea la cobertura amplia de la participación ciudadana con la confiabilidad técnica de los sensores fijos. Al cierre de esta evaluación, el sistema opera de extremo a extremo con datos reales: un sensor ESP32 publica sus lecturas vía MQTT al broker HiveMQ Cloud, el backend FastAPI las ingiere y las persiste en la base de datos Supabase con PostGIS, y el frontend Vue 3 consume esa información a través de la API REST para mostrarla en el heatmap interactivo y en las vistas diferenciadas por rol (vecino, funcionario municipal y administrador). Los beneficios esperados de la solución son dotar a la municipalidad de un mapa de ruido vivo y permanentemente actualizado, empoderar al vecino con datos objetivos sobre su entorno acústico, reducir la dependencia de las denuncias como único mecanismo de detección y generar series históricas que permitan evaluar tendencias y el impacto de intervenciones a lo largo del tiempo. Los próximos pasos del semestre contemplan ampliar la cobertura de pruebas automatizadas, refinar el dashboard municipal con base en la retroalimentación recibida, completar la documentación operacional del despliegue y preparar la presentación final del piloto funcional para la Municipalidad de Maipú, dejando además sentadas las bases arquitectónicas —particionamiento mensual de la tabla de lecturas y agregación precomputada vía pg_cron— para que el sistema pueda escalar a una red de sensores más extensa sin necesidad de reescritura.

Bibliografía

Organización Mundial de la Salud. (2021). *WHO global air quality guidelines: particulate matter (PM_{2.5} and PM₁₀), ozone, nitrogen dioxide, sulfur dioxide and carbon monoxide*. <https://iris.who.int/server/api/core/bitstreams/4c6945b7-497a-4043-9e6f-98d6774b3b61/content>

Superintendencia del Medio Ambiente. (7 de agosto de 2024). *SMA publica nueva guía para tramitar Programas de Cumplimiento por infracciones a la Norma de Ruido*. Recuperado el 5 de abril de 2026 de <https://portal.sma.gob.cl/index.php/sma-publica-nueva-guia-para-tramitar-programas-de-cumplimiento-por-infracciones-a-la-norma-de-ruido/>

Ministerio del Medio Ambiente. (2024). *Mapas de ruido*. Ruido Ambiental. Recuperado el 5 de abril de 2026 de <https://ruido.mma.gob.cl/mapas-de-ruido/>

Superintendencia del Medio Ambiente. (s. f.). *Infraestructura de Datos Espaciales (IDE)*. Recuperado el 5 de abril de 2026 de <https://ide.sma.gob.cl/>

Ministerio del Medio Ambiente. (s. f.). *Redes de monitoreo en línea*. Sistema de Información Nacional de Calidad del Aire. Recuperado el 5 de abril de 2026 de <https://sinca.mma.gob.cl/index.php/redes>